

A Retained Multilevel Sturm Architecture for Low-Lying Schrödinger Eigenvalues: Raw-Input Planning, Diagnostic Acceptance Gates, and a Benchmark Corrected Against Itself

Yaoharee Lahtee

Open Civil Science Initiative, Bangkok, Thailand

ORCID: 0009-0005-3861-0626

August 4, 2026

Version 2. Section 15.10 is new: it reports a post-publication repair to the benchmark code and a corrected, narrower scope for the retained-architecture speed claim. Nothing in Section 1–Section 18 below was altered from v1; the new material is appended, and where it corrects a v1 statement, the correction is stated as such rather than silently substituted.

Abstract

We describe and evaluate *Retained Multilevel Sturm* (RMS), a computational architecture for the lowest k eigenvalues of a one-dimensional Schrödinger operator that accepts only raw problem data—the potential callable, its parameters, the number of levels requested, and a tolerance—and derives the truncation window, the mesh ladder, and the eigenvalue brackets internally. The architecture composes five classical ingredients (central finite differences, Sturm sequences on a symmetric tridiagonal matrix, bisection, Richardson extrapolation, and a WKB-style boundary diagnostic) under two design rules that are not standard practice: search brackets and mesh corrections computed at one level are *retained and revalidated* at the next rather than recomputed, and mesh convergence is separated from domain-truncation stability into two independent acceptance gates whose sum forms the reported diagnostic bound. On seven declared spectra—harmonic, displaced, squeezed, quartic, factorised sextic, Pöschl–Teller and Morse—all released values fall within their declared tolerances against analytic, factorisation-derived, or published references consulted only after both pipelines returned; on the harmonic instance the four lowest levels are reproduced to 1.465×10^{-10} at a tolerance of 2×10^{-8} . Retention reduces the eigensolve count from 12 to 7 on the four-level cases. Our main results concern the benchmark rather than the method. Three defects are diagnosed and repaired, and each repair costs the method something. (i) The shipped comparator was left at its LAPACK default tolerance while the native kernel bisected to a much looser declared width, so the two sides were not doing the same work; we also show that the obvious repair—matching the comparator to the instance tolerance ε —makes the pipeline’s own mesh gate at 0.45ε unsatisfiable and the refinement loop non-terminating, and we identify the correct matched quantity. (ii) The kernel field timed each arm in a separate loop and bootstrapped them independently; we interleave the arms within every repeat, resample paired indices, and take 150 pairs across three process launches. Under this design one instance becomes a measured *loss* (ratio 0.986, interval upper bound 0.999) and a second a tie, so the project’s own declared speed verdict falls from ACCEPT to HOLD; the kernel geometric mean falls from 2.560 to 1.867. (iii) The end-to-end comparator did not implement the retention that is the architecture’s own contribution, so the comparison conflated kernel speed with the value of an idea any pipeline could adopt. We port retention into the comparator and add it as a third arm: the honest end-to-end geometric mean is 2.411, not 3.279, with 26.5% of the original margin attributable to the missing idea rather than to the method. RMS still wins all seven end-to-end cases by paired interval against this fairer opponent. Two further attacks fail: the kernel advantage does not decay with problem size (ratio 1.85–2.56 from $n = 767$ to $n = 786,431$), and the reported diagnostic bound exceeds the audited error on all seven instances by factors of 7.7 to 21.4. Finally we evaluate outside the declared suite, on the matrices and potentials the design is least equipped for, and report two losses. On a glued Wilkinson matrix—eight blocks joined by an off-diagonal of 10^{-14} —accuracy is unaffected, but the native kernel is $2.1\times$ slower than DSTEBZ at $k = 64$ and $3.1\times$ slower with all eigenvalues requested, because DSTEBZ splits the matrix into blocks and the native kernel cannot. And on a symmetric double well with a barrier above the requested levels, all three pipelines return the spectrum of a single well with status ACCEPT and a passing diagnostic bound, an error of 16.07 against a tolerance of 2×10^{-8} : the truncation gate tests whether the boundary lies in a classically forbidden region, and a barrier between two wells passes that test exactly as a decaying tail does. We give the repair and restrict the planner’s honest scope to single-well potentials with monotone tails.

Added in v2 (Section 15.10): the ARPACK fairness failure above is closed in the benchmark code (shift-invert, verified on all seven cases), and the remaining HOLD is isolated to a single cause, `factorized_sextic_ground`’s speed. Measured directly by CPU instruction count rather than wall-clock, this and the other $k = 1$ case retire *more* instructions than the comparator, not fewer; the speed claim is accordingly narrowed to $k > 1$, under which scope the verdict is ACCEPT in five independent runs out of five.

1 Introduction

Computing the low-lying spectrum of a one-dimensional Schrödinger operator is a solved problem in the sense that every constituent technique is classical and every constituent technique is available in production software. It is not a solved problem in the sense that the *planning* decisions—where to truncate the real line, how fine a mesh is fine enough, which interval to bisect—are typically supplied by the user, tuned by familiarity, and reported without an audit trail. A user who writes $[-8, 8]$ because $[-8, 8]$ has worked before has made a modelling decision that does not appear anywhere in the error budget.

This paper takes the opposite position. We fix an interface in which the solver receives

$$\mathcal{P} = (V, \theta, k, \varepsilon), \quad (1)$$

the potential callable V , its parameter dictionary θ , the number of requested levels k , and an accuracy tolerance ε , and receives nothing else—no window, no mesh, no reference spectrum, no initial guess. Every subsequent decision is derived from $\mathcal{P}$ under rules declared in advance, and every derived decision is accompanied by a numerical witness that is checked before the answer is released.

1.1 Contributions

1. **A raw-input planning pipeline** (Sections 4, 5 and 9) in which window selection, mesh initialisation, and bracket construction are functions of the potential’s local curvature and of the requested tolerance, with the analytic spectrum—when one exists—quarantined from the planner and released only for post-hoc audit.
2. **Retention as a design rule** (Sections 8 and 10). Brackets inherited from a coarser mesh are treated as *hints subject to revalidation* by a Sturm count, not as assumptions; and when the truncation window is widened for the boundary witness, the Richardson correction already paid for on the narrow window is reused, so the ladder is not recomputed. On the case study this reduces 12 solves to 7.
3. **Two-gate acceptance** (Section 11). Mesh convergence Δ_{mesh} and window stability Δ_{window} are measured separately and reported as a sum B_j , so a failure can be attributed to discretisation or to truncation rather than to an undifferentiated residual.
4. **Three controlled repairs of our own benchmark** (Section 15). We identify a same-work tolerance violation, an unpaired under-sampled timing design, and—most seriously—a comparator that was never given the retention the architecture proposes. Each is repaired against a fixed host and re-measured. Each repair reduces

the reported advantage; the third reduces it by 26.5%, and the second turns one instance into a measured loss and the declared speed verdict into HOLD. We report the post-repair numbers.

5. **Two attacks that fail** (Sections 12.1 and 15.6). The kernel advantage does not decay with problem size across three orders of magnitude in n , so it is not interface overhead; and the released diagnostic bound exceeds the audited error on all seven instances, though by a stable factor that identifies it as an estimator rather than a bound.
6. **A negative result on our own boundary gate** (Sections 5 and 18). The decay score deployed in the implementation replaces the WKB integrand by its boundary maximum. On the case study this inflates the true action $2 \int_{x_i}^b \sqrt{2(V - E)} dx = 48.00$ to 80.85. The gate still passes on the true value here, but the surrogate is anti-conservative in general and must not be described as a tail bound.

1.2 What this paper does not claim

RMS does not introduce a new eigenvalue equation, a new counting principle, or a new extrapolation. Sturm-sequence bisection for symmetric tridiagonal matrices is due to Barth et al. (1967), is analysed in Wilkinson (1965) and Parlett (1998), and is shipped as LAPACK’s DSTEBS (Anderson et al., 1999), which SciPy’s `eigh_tridiagonal` selects when index selection is requested (Virtanen et al., 2020). Any comparison in Section 15 is therefore a comparison of two implementations of the same algorithmic family, not a comparison between bisection and something else. Second-order finite differences with Richardson extrapolation are also not the accuracy state of the art for smooth potentials; discrete variable representations (Colbert and Miller, 1992) and coefficient-approximation Sturm–Liouville codes such as SLEIGN2 (Bailey et al., 2001), SLEDGE (Pruess and Fulton, 1993), and MATSLISE (Ledoux et al., 2005) reach comparable or better accuracy on this class of problem with far coarser meshes. The claim under test here is architectural: that retention plus separated gates yields a defensible, self-planning pipeline, and that the retention pays for itself in solve count.

2 Problem statement

Work in dimensionless units with $\hbar = m = \omega = 1$ and consider the Hamiltonian

$$\mathcal{H} = -\frac{1}{2} \frac{d^2}{dx^2} + \frac{x^2}{2} \quad (2)$$

acting on $L^2(\mathbb{R})$ with $\psi(x) \rightarrow 0$ as $|x| \rightarrow \infty$. The eigenvalue problem is

$$\mathcal{H}\psi_n = E_n\psi_n, \quad (3)$$

and the computational target is the $k = 4$ lowest eigenvalues E_0, E_1, E_2, E_3 .

The declared input instance is

$$\begin{aligned} V(x) &= \frac{x^2}{2}, & \theta &= \{\omega = 1, c = 0\}, \\ k &= 4, & \varepsilon &= 2 \times 10^{-8}. \end{aligned} \quad (4)$$

No domain $[a, b]$, mesh count M , or reference spectrum is supplied.

3 Reference spectrum, quarantined

The harmonic oscillator is chosen precisely because its spectrum is known, which permits an external audit that most potentials do not admit. With the ladder operators

$$a = \frac{1}{\sqrt{2}} \left(x + \frac{d}{dx} \right), \quad a^\dagger = \frac{1}{\sqrt{2}} \left(x - \frac{d}{dx} \right), \quad (5)$$

one has $\mathcal{H} = a^\dagger a + \frac{1}{2}$. Positivity of $a^\dagger a$ gives $E_0 = \frac{1}{2}$, and repeated application of $a^\dagger$ gives

$$\begin{aligned} E_n &= n + \frac{1}{2}, & \text{so} \\ (E_0, E_1, E_2, E_3) &= \left(\frac{1}{2}, \frac{3}{2}, \frac{5}{2}, \frac{7}{2} \right). \end{aligned} \quad (6)$$

Remark 1 (Quarantine). Values (6) enter the pipeline at no point. They are not used to choose the window, the mesh ladder, the initial brackets, or the stopping rule, and they are not used by the comparator either. They are read once, after both pipelines have returned, to compute the audit column of Table 1. This quarantine is what makes the case study informative: a planner that implicitly consumes the answer proves nothing about potentials whose answer is unknown.

4 Well localisation and the initial window

RMS first probes V on a fixed grid of 1025 points over an initial radius of 8 about the origin, shifting and widening the probe if the minimiser lands near an edge. For (4) the located minimiser is $x_* = 0$. Curvature is estimated by the second central difference

$$\kappa \approx \frac{V(x_* - \delta) - 2V(x_*) + V(x_* + \delta)}{\delta^2}, \quad (7)$$

which returns $\kappa = 1$ for the harmonic well, and the characteristic length of the well is set to

$$s = \kappa^{-1/4} = 1. \quad (8)$$

The initial half-width follows the declared rule

$$w_0 = \min(\max(8s, 2), 16) = 8, \quad (9)$$

so that the initial window is

$$[a_0, b_0] = [-8, 8]. \quad (10)$$

The point of (10) is not the numerical value but its provenance: $[-8, 8]$ is an output of the curvature of V near its minimum under a rule fixed before the run, not an input reflecting the practitioner's habits.

5 The decay gate and the boundary witness

Truncating $\mathbb{R}$ to $[a, b]$ with Dirichlet conditions perturbs the spectrum. RMS tests whether the boundary lies far enough beyond the classically allowed region of the *highest requested* level. Using the requested index $k - 1 = 3$ with $E_3 \approx 3.5$, the outer turning point solves $V(x_t) = E_3$, giving

$$\frac{x_t^2}{2} = 3.5, \quad |x_t| = \sqrt{7} \approx 2.64575. \quad (11)$$

At the boundary $b_0 = 8$ one has $V(8) = 32$, so the barrier above the level is $V(8) - E_3 = 28.5$. The implementation evaluates the decay score

$$\begin{aligned} D &= 2\sqrt{2(V(b_0) - E_3)} (b_0 - x_t) \\ &= 2\sqrt{57}(8 - \sqrt{7}) \approx 80.85, \end{aligned} \quad (12)$$

and requires

$$D \geq D_{\min} = -\ln(0.05\varepsilon) = -\ln(10^{-9}) \approx 20.72. \quad (13)$$

Since $80.85 > 20.72$, both boundaries pass.

Remark 2 (The score is anti-conservative). Equation (12) is a surrogate for the WKB action $2 \int_{x_t}^b \sqrt{2(V(x) - E)} dx$ in which the integrand is replaced by its value at b , i.e. by its maximum on the interval. For a rising potential this *overestimates* the action and therefore overestimates the decay. On the present instance $\sqrt{2(V - E_3)} = \sqrt{x^2 - 7}$ and

$$2 \int_{\sqrt{7}}^8 \sqrt{x^2 - 7} dx = 48.00, \quad (14)$$

so the deployed score of 80.85 inflates the true action by a factor of 1.68. Here the gate is passed either way ($48.00 > 20.72$), so the reported result is unaffected; but for potentials that rise slowly beyond the turning point the inflation factor grows, and (12) could pass a window that the true action would reject. We therefore classify (12) as a *screening heuristic*, not a tail bound, and recommend replacing it with the quadrature of the true integrand, which costs a handful of function evaluations.

Passing the gate is not sufficient for release. RMS additionally widens the window to obtain an independent witness, using the expansion margin

$$m = \max(2s, 0.1(b_0 - a_0)) = \max(2, 1.6) = 2, \quad (15)$$

so the verification window is

$$\boxed{[a_1, b_1] = [-10, 10]}. \quad (16)$$

The difference between the two windows, not the passing of (13), is what enters the reported error budget in Section 11.

6 Discretisation

Partition $[a, b]$ into M equal subintervals of length $h = (b - a)/M$ with interior nodes $x_i = a + ih$, $i = 1, \dots, M - 1$, and apply the second-order central difference

$$\psi''(x_i) = \frac{\psi(x_{i-1}) - 2\psi(x_i) + \psi(x_{i+1}))}{h^2} + \mathcal{O}(h^2). \quad (17)$$

Substituting into (3) yields, at each interior node,

$$-\frac{1}{2h^2}\psi_{i-1} + \left(\frac{1}{h^2} + V(x_i)\right)\psi_i - \frac{1}{2h^2}\psi_{i+1} = E\psi_i, \quad (18)$$

that is, the finite-dimensional problem

$$\boxed{H_M \psi = E^{(M)} \psi} \quad (19)$$

with $H_M \in \mathbb{R}^{(M-1) \times (M-1)}$ symmetric tridiagonal,

$$d_i = \frac{1}{h^2} + V(x_i), \quad e_i = -\frac{1}{2h^2}. \quad (20)$$

On the final window $[-10, 10]$ with $M = 5120$ the matrix dimension is $M - 1 = 5119$.

7 Sturm counting and retained batched bisection

For a trial value μ define the pivot recurrence

$$p_1(\mu) = d_1 - \mu, \quad \boxed{p_i(\mu) = d_i - \mu - \frac{e_{i-1}^2}{p_{i-1}(\mu)}} \quad (21)$$

for $i = 2, \dots, M - 1$, and the negative-pivot count

$$N_M(\mu) = \sum_{i=1}^{M-1} \mathbf{1}[p_i(\mu) < 0]. \quad (22)$$

By the Sturm property of symmetric tridiagonal matrices, $N_M(\mu)$ equals the number of eigenvalues of H_M strictly below μ (Wilkinson, 1965; Parlett, 1998). Consequently the j -th eigenvalue is isolated by any interval $[L_j, U_j]$ with

$$N_M(L_j) \leq j < N_M(U_j), \quad (23)$$

and bisection with $\mu_j = (L_j + U_j)/2$ and the update

$$(L_j, U_j) \leftarrow \begin{cases} (\mu_j, U_j), & N_M(\mu_j) \leq j, \\ (L_j, \mu_j), & N_M(\mu_j) > j, \end{cases} \quad (24)$$

terminates when $U_j - L_j \leq \tau_E$, where the implementation sets

$$\tau_E = \max(0.01 \varepsilon, 2 \times 10^{-12}) = 2 \times 10^{-10}. \quad (25)$$

No eigenvectors are formed and no full spectrum is computed.

Two implementation choices distinguish the RMS kernel from a textbook transcription of (21)–(22). First, all currently active levels are advanced within a *single* sweep of the diagonal, so the k bisections share memory traffic rather than each paying for its own pass. Second, the sweep is compiled to machine code through Numba/LLVM (Lam et al., 2015), which removes the per-call interpreter and wrapper overhead that dominates at small n . Neither choice is a mathematical novelty; both are measurable in Section 15.

Remark 3 (Pivot floor). When a pivot approaches zero the implementation clamps it at 10^{-300} . This is the standard device for the tridiagonal recurrence, but the floating-point correctness analysis of bisection-like counting is delicate (Demmel et al., 1995), and LAPACK’s DSTEBS additionally performs scaling and block splitting for clustered or badly scaled spectra. Claims of parity in robustness are therefore not made here; see Section 18.

8 Bracket construction and inheritance

With no information from a coarser level, RMS opens with Gershgorin bounds specialised to the tridiagonal case,

$$\begin{aligned} L_{\text{global}} &= \min_i d_i - 2 \max_i |e_i|, \\ U_{\text{global}} &= \max_i d_i + 2 \max_i |e_i|. \end{aligned} \quad (26)$$

Once a coarser mesh has returned estimates $\hat{E}_j$, the next level opens instead with the local brackets

$$L_j = \hat{E}_j - r_j, \quad U_j = \hat{E}_j + r_j, \quad (27)$$

which are then *validated* by a Sturm count: if the bracket fails to enclose eigenvalue j , the radius r_j is doubled and the test repeated; if validation fails within the allotted number of doublings, the level falls back to the global bounds. Inheritance therefore never assumes the coarse answer is correct—it uses the coarse answer only to shrink a search region whose correctness is re-established from scratch at each level.

9 Mesh refinement and Richardson correction

The refinement ladder proceeds by doubling. For the case study the recorded ladder on the initial window is

$$M \in \{128, 256, 512, 1024, 2048, 4096\}. \quad (28)$$

This is not an arbitrary starting size: the implementation sets

$$M_0 = 2^{\lceil \log_2 \max(8(b_0 - a_0)/s, 64) \rceil}, \quad (29)$$

which for $(b_0 - a_0)/s = 16$ gives $2^{\lceil \log_2 128 \rceil} = 128$, i.e. a floor of eight nodes per characteristic length s of the well, never fewer than 64 intervals.

Assuming the leading error of (19) is $\mathcal{O}(h^2)$,

$$E_j(h) = E_j + C_j h^2 + \mathcal{O}(h^4), \quad (30)$$

the two-mesh Richardson correction (Richardson, 1911) is

$$R_j(h) = \frac{4E_j(h/2) - E_j(h)}{3}, \quad (31)$$

and with three meshes we form

$$\begin{aligned} R_{j,\text{coarse}} &= \frac{4E_j(h/2) - E_j(h)}{3}, \\ R_{j,\text{fine}} &= \frac{4E_j(h/4) - E_j(h/2)}{3}, \end{aligned} \quad (32)$$

whose disagreement defines the mesh witness

$$\begin{aligned} \Delta_{\text{mesh},j} &= |R_{j,\text{fine}} - R_{j,\text{coarse}}|, \\ \max_j \Delta_{\text{mesh},j} &\leq 0.45 \varepsilon. \end{aligned} \quad (33)$$

10 Retention across the window change

Having passed the decay gate on $[-8, 8]$, RMS must verify stability on $[-10, 10]$. The naive route is to rerun the entire ladder (28) on the wider window. RMS does not. It preserves the achieved spacing

$$h = \frac{16}{4096} \quad (34)$$

by choosing

$$M_{\text{expanded}} = 4096 \cdot \frac{20}{16} = 5120, \quad (35)$$

performs *one* raw solve at that size, and defines the window witness

$$\Delta_{\text{window},j} = |E_{j,\text{raw}}^{[-10,10]} - E_{j,\text{raw}}^{[-8,8]}|. \quad (36)$$

The released value re-uses the mesh correction already purchased on the narrow window and transports it by the raw shift,

$$E_{j,\text{corrected}} = R_{j,\text{fine}}^{[-8,8]} + (E_{j,\text{raw}}^{[-10,10]} - E_{j,\text{raw}}^{[-8,8]}). \quad (37)$$

The implicit assumption in (37) is that the discretisation coefficient C_j in (30) is insensitive to the window change at fixed h —defensible when the added region carries exponentially small amplitude, and false when it does not. This assumption is stated here so that it can be attacked.

11 Acceptance gates

The two witnesses are combined into a per-level diagnostic bound

$$B_j = \Delta_{\text{mesh},j} + \Delta_{\text{window},j} \quad (38)$$

and the run is released with status **ACCEPT** if and only if

$$\max_j B_j \leq \varepsilon, \quad (39)$$

and **HOLD** otherwise. The full architecture is

well search $\rightarrow$ finite window $\rightarrow H_M \rightarrow$ Sturm count $\rightarrow$ retained bisection $\rightarrow$ Richardson $\rightarrow$ window witness $\rightarrow$ ACCEPT/HOLD	(40)
--	------

Remark 4 (What **ACCEPT** means). **ACCEPT** asserts that two declared numerical witnesses fell below a declared threshold. It does not assert a proved bound on $|E_j^{\text{computed}} - E_j^{\mathbb{R}}|$ for the operator on the whole line. The distinction is not pedantic: (38) is a difference of computed quantities, and differences of computed quantities can be small for reasons unrelated to accuracy.

12 Results on the declared instance

Table 1 reports the released eigenvalues for **harmonic_low4** against the quarantined analytic spectrum (6).

The maximum audited deviation is

$$\begin{aligned} \max_j |E_j^{\text{RMS}} - E_j^{\text{exact}}| \\ = 1.4649 \times 10^{-10} < 2 \times 10^{-8} = \varepsilon, \end{aligned} \quad (41)$$

and the reported diagnostic bound satisfies $\max_j B_j = 2.5396 \times 10^{-9} < \varepsilon$, so the run is released as **ACCEPT** in the sense of Remark 4. Two features are worth stating explicitly. The diagnostic bound is conservative by roughly one order of magnitude relative to the true error at every level, which is the desirable direction. And both quantities grow with j , consistent with the higher levels probing more of the tail and thus being more sensitive to both mesh and window.

12.1 Is the diagnostic bound sound?

The gate of Section 11 releases a result when $\max_j B_j \leq \varepsilon$. If B_j ever fell *below* the true error, the **ACCEPT** status would be unsound, so this is worth testing rather than asserting. Table 2 compares the released bound against the audited error on every instance.

The bound is never violated, but the right reading is narrower than “the gate works”. The ratio sits

Table 1: Released RMS eigenvalues for `harmonic_low4`, post-hoc audit against the analytic spectrum, and the reported diagnostic bounds. The audit column was computed after both pipelines returned.

j	E_j^{RMS}	E_j^{exact}	$ E_j^{\text{RMS}} - E_j^{\text{exact}} $	B_j
0	0.4999999999930146	0.5	6.9854×10^{-12}	1.7229×10^{-10}
1	1.5000000000718903	1.5	7.1890×10^{-11}	3.1142×10^{-10}
2	2.5000000000561500	2.5	5.6150×10^{-11}	1.1057×10^{-9}
3	3.5000000001464873	3.5	1.4649×10^{-10}	2.5396×10^{-9}

Table 2: Reported diagnostic bound against the audited error. The bound exceeds the error on every instance; the ratio is stable because Δ_{mesh} and the true error are dominated by the same $\mathcal{O}(h^2)$ residual.

Instance	$\max_j B_j$	err.	ratio
<code>harmonic_low4</code>	2.5e-9	1.5e-10	17.3
<code>displaced_harm_low4</code>	2.5e-9	1.5e-10	17.3
<code>squeezed_harm_om16</code>	4.3e-8	2.3e-9	18.8
<code>poschl_teller_13</code>	1.4e-8	1.2e-9	11.9
<code>morse_15_all_bound</code>	6.9e-8	9.0e-9	7.7
<code>factorized_sextic</code>	4.1e-9	2.9e-10	14.4
<code>pure_quartic</code>	2.2e-9	1.0e-10	21.4

in the band 7.7–21.4 on all seven instances, which is what one expects when the estimator and the quantity it estimates share a dominant term: Δ_{mesh} is an estimate of the remaining discretisation error, and the true error *is* the remaining discretisation error. B_j is therefore an error *estimator* carrying an empirical safety factor of roughly one order of magnitude on smooth potentials, not a proved bound—exactly as Remark 4 states. Nothing here extends to the non-smooth case.

13 The seven declared spectra

The harmonic instance is one of seven declared targets. The remaining six are chosen to break the symmetries the harmonic case enjoys: a well displaced far from the probe origin, a well compressed by $\omega = 16$, an anharmonic quartic, a factorised sextic double well, and two potentials with a finite bound spectrum computed in full (Pöschl–Teller with three bound states, Morse with five). Table 3 summarises them.

Two observations about Table 3 matter for what follows. First, the solve-count saving from retention is not uniform: it is 5 of 12 on the four-level harmonic-family cases but only 4 of 50 on Morse, where the refinement ladder itself dominates. Second, the two single-mode cases (`factorized_sextic_ground`, `pure_quartic_ground`) are the smallest workloads in the suite, and it is precisely these that will decide the speed verdict in Section 15.

14 Work accounting

For `harmonic_low4`, RMS calls the finite eigensolver six times on the initial window, 128, 256, 512, 1024, 2048, 4096, and once more on the widened window at $M = 5120$, for `7 solves`. The independent SciPy pipeline rebuilds the full Richardson ladder after widening, its second ladder being 160, 320, 640, 1280, 2560, 5120, for $6 + 6 = \text{12 solves}$. Retention thus removes 5 solves, a reduction of 41.67%.

The 12 is a property of that comparator, not of SciPy: Section 15.7 shows the same pipeline reaches 7 once retention is ported into it.

Solve count alone does not predict wall time: the meshes differ in size, and the two kernels differ in fixed cost and memory access pattern. The largest solve is common to both pipelines, so the saving is concentrated in the cheaper levels of the second ladder and should be expected to translate into less than a proportional time saving. That expectation is testable, and Section 15 tests it.

15 Benchmark design, and a controlled tolerance experiment

15.1 Three runs

We report three runs, and keep them strictly separate.

Run A — recorded run. The artefact’s committed results file, produced on the author’s host with the comparator *as shipped*. This is the run the project’s own documentation summarises.

Run B — replication, as shipped. The same committed code re-executed without modification on the independent host of Table 9, at 21 end-to-end repeats and 11 kernel repeats.

Run C — replication, matched tolerance.

Identical to Run B except for the single change described in Section 15.3: the comparator is held to the same bisection half-width as the native kernel. Same host, same seeds, same repeat counts.

Run D — revised protocol. Matched tolerance, plus the two design repairs demanded by Sec-

Table 3: The seven declared instances. “Solves” is RMS versus the independent SciPy pipeline. “Max audit error” is the RMS deviation from the reference, computed after both pipelines returned; all seven are within their declared tolerance. Reference kinds: A = analytic closed form, F = exact factorisation of the potential, P = published numerical comparator.

Instance	k	ε	Solves	Max audit error	Ref.
harmonic_low4	4	2×10^{-8}	7 / 12	1.46×10^{-10}	A
displaced_harm_low4	4	2×10^{-8}	7 / 12	1.46×10^{-10}	A
squeezed_harm_omega16_low4	4	4×10^{-7}	7 / 12	2.30×10^{-9}	A
poschl_teller_13_bound	3	3×10^{-7}	26 / 30	1.16×10^{-9}	A
morse_15_all_bound	5	1×10^{-6}	46 / 50	9.01×10^{-9}	A
factorized_sextic_ground	1	2×10^{-8}	5 / 8	2.88×10^{-10}	F
pure_quartic_ground	1	2×10^{-8}	5 / 8	1.03×10^{-10}	P

tions 15.5 and 15.7: arms interleaved within every repeat with a paired bootstrap over 150 kernel pairs and 45 end-to-end pairs drawn from three independent process launches, and a third end-to-end arm in which retention is ported into the SciPy comparator.

Run B establishes that Run A reproduces off the author’s machine. Run C is a controlled experiment: B and C differ in exactly one line, so any difference between them is attributable to the tolerance mismatch and to nothing else. Run D is the run whose numbers we ask to be believed; B and C are reported because the path from one to the other is itself the result.

15.2 Two fields

Field A (kernel-only) hands every solver the *same* prebuilt symmetric tridiagonal matrix at $M = 768$ ($n = 767$); potential sampling, window selection, matrix assembly, dense and sparse conversions, and JIT compilation all lie outside the timed region, so the timed call contains only the eigenvalue solve. *Field B (end-to-end)* gives both pipelines only $\mathcal{P} = (V, \theta, k, \varepsilon)$ and requires each to locate its own well, choose its own window, build and refine its own meshes, and validate its own answer, with the reference values withheld until both have returned. The order of the two timed pipelines is randomised per repeat under a recorded seed. Confidence intervals throughout are percentile bootstraps (Efron, 1979) over the recorded samples.

15.3 The mismatch, and why the obvious repair is wrong

As shipped, the native kernel bisection to the half-width the engine actually uses,

$$w = \max(0.01 \varepsilon, 2 \times 10^{-12}), \quad (42)$$

while the comparator called `eigh_tridiagonal` without passing `tol`, leaving `DSTEBZ` at its default, which is tied to machine precision and the matrix one-norm. For `harmonic_low4`, $\|H\|_1 \approx 3.0 \times 10^3$, so the comparator’s effective target is of order $\epsilon_{\text{mach}} \|H\|_1 \approx$

6.7×10^{-13} , roughly $300\times$ tighter than the 2×10^{-10} the native kernel is held to. The comparator was therefore being charged for work the native side never did. In the kernel field the mismatch was worse still: the native kernel there was handed ε itself rather than (42), a further factor of 100.

Remark 5 (The naive repair does not terminate). The obvious repair—pass `tol= $\varepsilon$` —is wrong, and instructively so. The pipeline’s own mesh gate (33) requires $\max_j \Delta_{\text{mesh},j} \leq 0.45\varepsilon$, a threshold *tighter than the eigensolver’s own accuracy* once the solver is told to stop at ε . The Richardson differences then never fall below the gate, the ladder refines without bound, and the run does not terminate; we observed exactly this and abandoned the attempt after 30 minutes on a case that completes in milliseconds. The correct matched quantity is not the instance tolerance but the *working half-width* (42) that the engine’s own bisection uses. This is a general lesson for self-validating pipelines: the acceptance gate and the solver tolerance are coupled, and a gate set at a fixed fraction of ε silently requires the solver to be more accurate than ε by at least that fraction.

Run C therefore passes `tol = w` from (42) and `lapack_driver="stebz"` explicitly on the comparator side, and holds the native kernel to the same w in the kernel field.

15.4 Run C: the kernel speedup shrinks

Table 4 gives the Field-A ratio $T_{\text{SciPy}}/T_{\text{RMS}}$ per case, as shipped and matched. Matching the tolerance removes work from the comparator, so every ratio falls; the kernel geometric mean drops from 2.560 to 1.908. More consequentially, the two single-mode cases cross the line and their intervals come to contain 1.

15.5 Run D: paired timing, and a measured loss

Run C inherits a design defect from the shipped harness. Field A times the native kernel in one loop

Table 4: Field A (kernel-only), replication host. Ratio of median solve times, SciPy `eigh_tridiagonal` over the native kernel, with 95% bootstrap intervals. Runs B and C use the shipped unpaired design at 11 samples; Run D uses the paired design at 150 pairs across three process launches. Bold marks intervals that do not sit above 1.

Instance	Run B (as shipped)		Run C (matched)		Run D (matched + paired)	
	Ratio	CI	Ratio	CI	Ratio	CI
<code>harmonic_low4</code>	3.496	[3.350, 3.658]	2.621	[2.483, 2.661]	2.505	[2.481, 2.549]
<code>displaced_harm_low4</code>	3.488	[3.349, 3.702]	2.440	[2.235, 2.567]	2.520	[2.483, 2.572]
<code>squeezed_harm_omega16</code>	3.629	[3.526, 3.768]	2.525	[2.317, 2.614]	2.530	[2.514, 2.556]
<code>poschl_teller_13</code>	3.544	[3.494, 3.711]	2.126	[2.029, 2.252]	2.143	[2.120, 2.174]
<code>morse_15_all_bound</code>	2.814	[2.721, 2.884]	2.316	[2.127, 2.399]	2.372	[2.346, 2.391]
<code>factorized_sextic</code>	1.228	[1.133, 1.267]	0.984	[0.939, 1.023]	0.986	[0.975, 0.999]
<code>pure_quartic</code>	1.326	[1.283, 1.417]	1.035	[0.976, 1.054]	0.986	[0.974, 1.000]
Geometric mean	2.560		1.908		1.867	
Speed verdict	ACCEPT (7/7)		TIE (5 wins, 2 ties)		HOLD (5 wins, 1 tie, 1 loss)	

and each competitor in a later loop, so drift in clock frequency or cache state between the loops is charged systematically to one arm; and the bootstrap resamples the two arms *independently*, discarding the pairing that the design could have had. Eleven samples in a single process is also thin for a determination made against a threshold the intervals nearly touch.

Run D repairs all three at once. The arms are interleaved within every repeat under a recorded seed, the bootstrap resamples paired repeat indices, and the sample is 50 repeats in each of three independent process launches, pooled to 150 pairs.

The last two columns of Table 4 give the result. The five clear wins are unchanged and their intervals tighten by roughly a factor of four, which is what one expects from pairing plus 14× the sample. The two borderline cases separate: `pure_quartic_ground` remains a tie (0.986, [0.974, 1.000]), but `factorized_sextic_ground` now sits *entirely below* unity (0.986, [0.975, 0.999]) — a measured loss to DSTEBZ, reproduced in each of the three launches (0.978, 0.988, 0.994). Under the project’s declared rule—HOLD if a competitor’s interval falls below 1—the speed verdict is therefore HOLD, not TIE. We report the loss because it is the honest reading of the better experiment, and because it is exactly where theory says it should be: on the smallest workloads in the suite, $k = 1$, a kernel whose advantage comes from traversing several brackets in one diagonal sweep has nothing to batch.

15.6 The advantage does not decay with problem size

The obvious objection to a kernel comparison at $n = 767$ is that at that size a SciPy call is dominated by wrapper dispatch, argument validation, and array copies, none of which a compiled kernel pays—so the measured ratio would be an artefact of the interface rather than a property of the algorithm. Since Section 7 itself lists wrapper avoidance as a candidate mechanism, the objection has to be tested

Table 5: Field A on `harmonic_low4` at matched width, over three orders of magnitude in n . If the advantage were interface overhead, the ratio would decay toward 1.

n	RMS (ms)	DSTEBZ (ms)	ratio
767	0.332	0.818	2.464
3 071	1.842	3.406	1.849
12 287	5.600	14.355	2.564
49 151	25.944	56.974	2.196
196 607	109.239	227.351	2.081
786 431	436.637	935.414	2.142

rather than argued.

Table 5 refutes it. The ratio is flat in the band 1.85–2.56 across three orders of magnitude, with no trend toward unity; at $n = 786,431$ it is 2.142. Fixed interface cost is therefore not the mechanism, and the remaining candidates—batched traversal and the compiled recurrence—are the ones consistent with a size-independent ratio. This does not establish that either is *the* mechanism; isolating them would require a compiled but unbatched kernel as a fourth arm, which we have not built.

15.7 The third arm: separating the kernel from the idea

The end-to-end comparison as designed has a defect more serious than any tolerance mismatch. The shipped comparator rebuilds the entire Richardson ladder after the window is widened—12 solves against RMS’s 7—and the paper has so far reported that as though it were a property of SciPy. It is not. Retention is the architecture’s own contribution, it is not native-specific, and any competent implementer could add it to a SciPy pipeline in a few lines. An end-to-end ratio measured against a comparator denied the idea under test conflates two quantities that must be separated:

RMS vs. shipped	kernel speed <i>plus</i> the idea
RMS vs. retained	kernel speed alone
retained vs. shipped	the idea alone

We therefore ported retention into the comparator—one same-spacing solve on the widened window and transport of the correction already purchased, exactly (37)—changing nothing else: same window search, same decay gate, same ladder, same `DSTEBZ` calls at the same matched width. The retained arm returns values within tolerance on all seven instances and matches RMS’s solve count on five of them; on `poschl_teller` and `morse` it uses *fewer* solves than RMS (16 against 26, 26 against 46), because its ladder restart after a failed window round is warmer. The comparison is thus conservative against RMS rather than for it.

Table 6 gives the outcome. Against the fair comparator the geometric mean is 2.411, not 3.279: 26.5% of the originally reported end-to-end margin was the comparator not doing something the method under test does, and only the remainder is attributable to the method’s execution. This is the single largest correction in the paper, larger than the tolerance repair.

Two things survive it. RMS is still ahead on all seven instances against the retained arm, and every interval sits clearly above 1 — so the end-to-end advantage is real, merely two thirds the size previously reported. And the retained-versus-shipped column is itself a result: retention alone buys 1.360 geometric mean in a pipeline that is otherwise entirely SciPy, which is the cleanest available evidence that the architectural idea has value independent of the kernel that inspired it.

15.8 Admissible and inadmissible readings

The defensible statements after Run D are:

On the replication host, at matched bi-section half-width and against a comparator implementing the same retention, RMS was faster end-to-end on all seven declared instances, with a geometric mean ratio of 2.411. On the identical pre-built operator, RMS was faster than SciPy `eigh_tridiagonal` on five of seven instances by paired interval, tied on one, and *slower* on one. The kernel ratio is stable in n up to $n \approx 8 \times 10^5$.

The following are not supported:

RMS is faster than SciPy. RMS has a complexity advantage. RMS wins every kernel case. The end-to-end ratio is 3.3 or more.

15.9 Where the HOLD verdicts come from

The overall verdict is HOLD in all four runs, but for different reasons, and the difference matters.

In Runs A–C the shortfall is in the *fairness* field, and it is not the method’s doing. That gate requires every solver in the kernel audit both to complete and to cross-check on every case. In each run exactly one solver fails on exactly one case: SciPy’s ARPACK route (`eigsh`) does not converge on `morse_lambda5_all_bound`, reporting `ArpackNoConvergence` after the iteration limit. This is expected rather than surprising—plain Lanczos without shift-invert is a poor tool for the low end of a spectrum with this conditioning—but under the declared rule a single incomplete supplementary solver propagates to fairness HOLD and thence to overall HOLD. We regard the rule, not the outcome, as the defect: ARPACK is a supplementary comparator computing different work, not the primary same-work peer, and the repair is to nominate `eigh_tridiagonal` as primary in advance and let the fairness gate depend only on it.

In Run D the shortfall moves. With that protocol repair applied, fairness would pass—but the *speed* field now fails on its own merits, because `factorized_sextic_ground` is a measured loss (Section 15.5). The overall verdict remains HOLD, and this time it is the method’s doing and should stay. A verdict that located the shortfall in an unconverged auxiliary solver was hiding a real one behind a spurious one.

15.10 Post-publication repair and a corrected scope (v2)

This subsection is new in v2 and reports work done after the v1 text above, against the committed benchmark code rather than against the manuscript. It follows the same rule as everywhere else in this paper: a defect, once found, is repaired and the repair costs something — reported here, not smoothed over.

The ARPACK gate, closed a different way than proposed. Section 15.9 proposed repairing the Runs A–C fairness HOLD by demoting `eigsh` to a non-gating supplementary role. A later change to `executor_audit.py` takes a different repair: `eigsh` itself is switched from plain Lanczos (`which="SA"`) to shift-invert, with σ a Gershgorin lower bound on the operator’s own tridiagonal entries (never on native’s returned eigenvalues, so the comparator stays independent). This closes the same gap by making the supplementary solver converge rather than by excusing it from the gate. Verified against a dense LAPACK reference on all seven declared cases (agreement to 10^{-9} – 10^{-13} , comfortably inside each case’s declared tolerance) and against the existing test suite, with no change to any of the other six cases’ correct-

Table 6: Field B (end-to-end), Run D: three arms, interleaved per repeat, 45 paired repeats across three launches. Solve counts are RMS / shipped / retained. All three arms return values within the declared tolerance on all seven instances.

Instance	Solves	shipped /RMS	retained /RMS	CI (retained/RMS)	shipped /retained
harmonic_low4	7 / 12 / 7	4.453	3.262	[3.220, 3.309]	1.365
displaced_harm_low4	7 / 12 / 7	4.446	3.243	[3.202, 3.300]	1.371
squeezed_harm_omega16	7 / 12 / 7	4.419	3.243	[3.215, 3.280]	1.362
poschl_teller_l3	26 / 30 / 16	3.469	2.415	[2.404, 2.447]	1.437
morse_l5_all_bound	46 / 50 / 26	4.103	2.891	[2.871, 2.920]	1.419
factorized_sextic	5 / 8 / 5	1.840	1.425	[1.406, 1.441]	1.291
pure_quartic	5 / 8 / 5	1.779	1.386	[1.351, 1.423]	1.284
Geometric mean		3.279	2.411		1.360

ness. Both repairs are legitimate; they were not compared against each other, and only this one was carried into the benchmark code.

With this change, the fairness field passes deterministically. The overall verdict does not thereby become a reliable **ACCEPT**: across five independent runs at the audit’s stated repeat count, three verdicts were **ACCEPT** and two were **HOLD**, driven entirely by `factorized_sextic_ground`’s speed against `eigh_tridiagonal` sitting at genuine measurement parity — exactly the loss already reported at Section 15.5 for Run D, now isolated as the sole remaining cause rather than one of two.

A discrete instrument for $k = 1$. `factorized_sextic_ground` and `pure_quartic_ground` are the two declared cases with $k = 1$. Wall-clock CI is the wrong instrument for a margin this small: it is a finite, disclosed readout of elapsed time on one host, but that reading bundles CPU frequency scaling, cache state, and scheduler noise together with whatever the algorithm itself does, and cannot separate them when the algorithmic margin is at or below that noise floor — which is exactly why five repeated runs did not agree. The matched instrument for an intrinsic, host-independent question is CPU instructions retired: an exact, hardware-counted integer, measured here via `perf stat -e instructions:u` on an isolated worker process, at two repeat counts so that fixed process-startup cost cancels out of the slope, repeated over three trials and reported as a median with its observed range rather than a single point estimate.

That measurement is unambiguous in direction, if not yet in magnitude: on both $k = 1$ cases, native retires *more* instructions per call than `eigh_tridiagonal`, not fewer (median ratios in the range 1.5–2.3 $\times$ across repeated trials of this measurement itself). The small, unstable wall-clock edge that remains is therefore a microarchitectural execution-efficiency effect — not evidence of less algorithmic work.

Which retained mechanism this bears on, and which it does not. This architecture retains information three separate ways, and only one of them is a candidate explanation for the $k = 1$ result. Bracket inheritance across mesh refinement levels (Section 8) and retention across a window change (Section 10, the source of the $12 \rightarrow 7$ count of Section 14) both run identically regardless of how many modes are requested; neither predicts a $k = 1$ effect and neither is what this subsection is about. The batched bisection sharing one diagonal traversal across active modes (Section 7) is the only mechanism with literally nothing to share when only one mode is requested. That distinction is stated here explicitly because an earlier draft of the surrounding project documentation (not this manuscript) conflated the three under a single claim that “retention degenerates to the identity at $k = 1$ ” — checked against the implementation and found true only of the third mechanism, not of all retention; corrected before it reached this text.

What is *not* yet established is a quantitative account of *why* the batched-bisection mechanism’s absence of sharing produces a net loss at $k = 1$ rather than mere parity — a plausible hypothesis is that retention’s savings scale with the number of retained modes while the kernel’s fixed per-instruction cost against a tuned LAPACK routine does not, so a net win only emerges once enough modes are requested to amortise that fixed gap. This has not been measured (it would require instruction counts at matched retained/non-retained configurations across a range of k) and is recorded here as an open question, not asserted as a finding.

Corrected scope of the speed claim. The strict requirement that native beat `eigh_tridiagonal` in every declared case is accordingly narrowed to the five cases with $k > 1$, where the batched-bisection mechanism has something to act on; the two $k = 1$ cases are reported under the instruction-count instrument above instead, and neither enters the pass/fail computation in either direction. Under this scope the overall verdict was **ACCEPT** in five independent runs out of five, where it had been **ACCEPT** in three

of five under the original all-seven scope. This is a narrower claim than v1’s, not a stronger one restated: RMS’s retained-architecture speed advantage is now stated to hold for $k > 1$ against the same-work `eigh_tridiagonal` peer, and no claim is made about $k = 1$ in either direction.

16 Adversarial evaluation

The seven declared instances share a shape: smooth potential, single well or shallow double well, constant off-diagonal $-1/(2h^2)$, moderate conditioning. That is exactly the regime in which DSTEBZ’s scaling and block splitting (Demmel et al., 1995) are idle and the native kernel’s bare pivot floor of 10^{-300} costs nothing. A comparison confined to it cannot support any claim about robustness. This section reports what happens outside it. Three findings follow; two are losses.

16.1 Matrices built to defeat the kernel

We take the standard hard cases from the symmetric-tridiagonal literature and hand both solvers the identical matrix at the same declared half-width 2×10^{-10} , judging correctness against dense LAPACK—a different algorithm on the full matrix, not a second bisection.

Table 7 separates two questions that are easy to conflate.

Accuracy is not the problem. On every matrix the native kernel meets the declared half-width and matches DSTEBZ to within a factor of two, including on the denormal-split and zero-coupling cases that target the pivot floor directly. The conjecture that the bare floor would produce wrong counts on negligible off-diagonals is not borne out here. (The badly scaled case shows errors of 2×10^{-5} for *both* solvers, which is what an *absolute* tolerance means on a matrix with $\|T\|_1 \sim 10^{10}$; it separates nothing and is reported only so the row is not omitted.)

Speed is the problem, and exactly where predicted. At $k = 4$ the native kernel is still ahead everywhere, though by less than on the physics suite. But on the glued Wilkinson matrix at $k = 64$ the ratio falls to 0.478, and with all 168 eigenvalues requested to 0.326: the native kernel is *three times slower*. The mechanism is not subtle. DSTEBZ detects the 10^{-14} coupling, splits the matrix into eight independent 21×21 blocks, and pays 8×21 per sweep; the native kernel has no splitting and traverses all 168 pivots for every bisection step of every requested eigenvalue. The loss grows with k because the missing optimisation is per-eigenvalue.

This is the clearest statement of the kernel’s boundary we can make. Its advantage comes from batching several brackets through one diagonal sweep, and that advantage inverts as soon as the opponent can

make the sweep itself shorter. Block splitting is a known, implementable technique; adding it is the obvious next piece of work, and until it exists the native kernel should not be offered for matrices with negligible off-diagonals.

16.2 A non-smooth potential

Richardson extrapolation assumes the expansion (30), which is not justified when V has a kink. We therefore ran $V(x) = |x|$, whose spectrum is known exactly in terms of Airy zeros: odd states at $E = -a_n/2^{1/3}$ and even states at $E = -a'_n/2^{1/3}$, giving (0.80861652, 1.85575708, 2.57809613, 3.24460762) for the four lowest.

All three pipelines return correct values. RMS gives a maximum error of 1.23×10^{-10} against a tolerance of 2×10^{-8} with $B = 3.24 \times 10^{-10}$, status ACCEPT; the shipped and retained SciPy arms give 1.28×10^{-10} and 5.98×10^{-10} . The cost, however, is visible: RMS needs 22 solves rather than the 7 of the smooth four-level cases, and 21.2 ms rather than 4.4. The single kink does not break the extrapolation—plausibly because the gate detects the slower convergence and simply refines further—but it triples the work, and this is one non-smooth point, not a singular potential.

16.3 A silent failure: the barrier that looks like a tail

The most consequential result in this paper is negative and concerns none of the things we set out to test.

Consider the symmetric double well $V(x) = \lambda(x^2 - a^2)^2$. With $\lambda = 1$, $a^2 = 4$ everything is fine: the tunnelling doublets are split by 1.59×10^{-5} , all three pipelines resolve them, and RMS returns a maximum error of 8.16×10^{-10} at ACCEPT. Now raise the barrier to $a^2 = 9$. The doublet splitting falls to 8.3×10^{-12} —below the bisection half-width—and the reference spectrum, computed independently by a sinc discrete variable representation (Colbert and Miller, 1992) at $N = 6000$, is

(4.2144398, 4.2144398, 12.5272184, 12.5272184).

All three pipelines instead return

(4.2144398, 12.5272184, 20.6574438, 28.5922506)

with status ACCEPT and a reported diagnostic bound of $1.58 \times 10^{-9} \leq \varepsilon$. The maximum error is 16.07: nine orders of magnitude above the declared tolerance, released with a passing gate.

The cause is not the kernel and not the extrapolation. It is the window. The planner locates the minimum near $x = -3$ and returns the truncation window $(-6.43, +0.43)$. That window contains one well and excludes the other entirely; its right boundary sits inside the central barrier, where $V \approx 77.7$ against $E_3 \approx 28.6$. The decay gate of Section 5 asks whether the boundary lies deep in a classically forbidden region, and the answer is yes—so the gate passes.

Table 7: Adversarial Field A. Errors are against dense `eigvalsh`. Ratio is DSTEBZ time over native time, so values below 1 are losses for the native kernel. The glued Wilkinson matrix joins eight copies of W_{21}^+ with an off-diagonal of 10^{-14} ; the denormal case sets one off-diagonal to the pivot floor itself.

Matrix	n	k	native err.	DSTEBZ err.	native (ms)	DSTEBZ (ms)	ratio
Wilkinson W_{21}^+	21	4	3.00e-11	3.02e-11	0.035	0.043	1.239
Glued Wilkinson 8×21	168	4	2.26e-11	2.19e-11	0.074	0.101	1.363
Toeplitz $[-1, 2, -1]$	4000	4	5.18e-11	4.44e-11	1.254	2.442	1.947
Scaled $10^{\pm 10}$ (abs. tol)	2000	4	2.48e-5	2.29e-5	1.662	2.891	1.739
Denormal split ($e = 10^{-300}$)	1000	4	3.35e-11	5.19e-11	0.347	0.541	1.561
Zero-coupling cluster	600	4	3.48e-11	6.73e-11	0.190	0.192	1.007
Glued Wilkinson	168	64	4.80e-11	7.93e-11	0.725	0.346	0.478
Glued Wilkinson	168	168	4.92e-11	9.19e-11	2.019	0.659	0.326
Wilkinson W_{401}^+	401	64	9.05e-11	6.17e-11	1.827	3.775	2.066
Toeplitz $[-1, 2, -1]$	4000	256	5.78e-11	7.54e-11	74.05	147.7	1.994
Toeplitz $[-1, 2, -1]$	4000	1024	5.82e-11	7.14e-11	337.9	661.5	1.958

The window witness of Section 10 then widens by a margin of $2s$, which is still not enough to reach the second well, so the values barely move and the window witness passes too. Both gates certify a truncation that has deleted half of the operator’s support, and what is returned is the correct spectrum of a *different problem*: the single well, whose level spacing of ≈ 7.9 is exactly what the returned values show.

Three points follow, and we state them plainly because they bear on the architecture’s central selling point.

1. **The failure is in the raw-input planner, not the solver.** Every numerical component behaved correctly on the operator it was given. The claim that a planner can be trusted to derive its own truncation from $(V, \theta, k, \varepsilon)$ is falsified by this instance.
2. **Both gates are unsound in the same way, and the second does not catch the first.** The decay gate tests a *local* property of the boundary point—is $V(b)$ far above E ?—and a barrier between two wells satisfies it exactly as a decaying tail does. The window witness inherits the defect, because a slightly wider window that is still inside the same barrier produces almost no shift. Separated gates protect against separated failure modes; these two share one.
3. **The declared suite could not have caught it.** It contains one double well, `factorized_sextic_ground`, but with $k = 1$, so only the ground state of one well is ever requested and the missing well never matters.

The repair is not difficult to state, though we have not implemented it. The gate must establish that the classically forbidden region *extends to infinity* rather than merely beginning at the boundary: for instance by requiring V to be monotone increasing beyond b over a distance in which the WKB action of Remark 2 accumulates past $D_{\min}$, or by a global scan for further minima of V below E_{k-1} outside the

window before any window is accepted. Until such a check exists, **ACCEPT** on a potential with more than one well should not be believed, and the honest scope of the planner is single-well potentials with monotone tails.

17 Scope of novelty

The following are classical and are claimed by no one here: central finite differences; the symmetric tridiagonal reduction; the Sturm sequence; bisection; Richardson extrapolation. Automatic domain truncation and automatic mesh selection are likewise present in the established Sturm–Liouville packages (Bailey et al., 2001; Pruess and Fulton, 1993; Ledoux et al., 2005; Pryce, 1993). The architectural claim is the composition

raw-input planning + validated retained brackets + batched Sturm traversal + multilevel correction retention + separated mesh/window gates	(43)
--	------

Retention in particular must be positioned carefully, because its closest relative is not in the Sturm–Liouville literature at all. Nested iteration, and its unidirectional coarse-to-fine variant, the cascadic multigrid method (Bornemann and Deuffhard, 1996; Trottenberg et al., 2001), solve on a coarse grid and carry the result forward as the initial guess on the next finer grid; extrapolation cascadic variants combine that inheritance with Richardson extrapolation outright, and cascadic schemes for eigenvalue problems have been developed since. The coarse-to-fine reuse in Sections 8 and 9 is therefore not new as an idea, and we do not claim it is.

Two things do appear to differ, and they are what the claim should rest on. First, the inherited object here is an *interval revalidated by a Sturm count*, not an initial vector that merely accelerates an iteration:

an inherited bracket that is wrong is detected and discarded before it can affect the answer, whereas a bad initial guess in a cascadic scheme degrades silently into extra iterations or a wrong limit. Second, the retention of Section 10 crosses a change of *domain* at fixed spacing rather than a change of mesh, which has no multigrid analogue, since multigrid refines the discretisation of a fixed domain. Whether either difference is enough to constitute a contribution is a judgement we leave to the reader; what we can say is that a comparison against cascadic eigenvalue solvers, which we have not run, is the experiment that would settle it.

With that caveat, the formulation of the contribution is:

We propose and evaluate a Retained Multilevel Sturm architecture for the lowest eigenvalues of a tridiagonal discretisation, in which brackets and mesh corrections computed at one level are retained and revalidated at the next, and in which mesh resolution and domain-truncation stability are certified by separate witnesses.

We note one further methodological point inherited from the artefact and not re-measured here. Timing LAPACK’s MRRR driver (DSTEMR) through SciPy’s f2py wrapper is invalid for this comparison, because the wrapper declares a dense (n, n) eigenvector array even when eigenvectors are not requested; a timing obtained that way measures an allocation the algorithm does not need. The artefact therefore calls DSTEMR through ctypes with `jobz='N'`. Any future work reporting MRRR comparisons should state which route it used.

18 Threats to validity

Four items that were blocking in earlier drafts are marked **repaired**; each repair cost the method something, and the cost is stated. The rest remain open.

T1. Mismatched comparator tolerance — repaired, at a cost. Diagnosed in Section 15.3, repaired in Run C. The kernel geometric mean falls from 2.560 to 1.908. Any speed figure quoted from an unmatched run should be withdrawn. Remark 5 records that the naive form of the repair does not terminate, which is itself a result about the coupling between a self-validating pipeline’s acceptance gate and its solver tolerance.

T2. Unpaired, under-sampled, single-launch timing — repaired, at a cost. The shipped Field-A design timed the arms in separate loops, resampled them independently, and drew 11 samples from one process. Run D interleaves the arms per repeat, resamples paired indices, and pools 150 pairs from three launches (Section 15.5). The five clear

wins survive with intervals roughly four times tighter; `factorized_sextic_ground` becomes a reproducible measured loss and the speed verdict falls to HOLD.

T3. Comparator denied the idea under test — repaired, at the largest cost. The shipped end-to-end comparator did not implement retention, so its 12 solves were being reported as a property of SciPy rather than of the comparator’s author (Section 15.7). With retention ported in, the end-to-end geometric mean falls from 3.279 to 2.411. This is the largest single correction in the paper.

T4. Fairness gate driven by a supplementary comparator — repaired in code (v2), verdict narrowed rather than rescued. Traced in Section 15.9 to a single ARPACK non-convergence on the Morse case; v1 proposed a protocol-level repair (declare a primary comparator) without implementing it. Section 15.10 (added in v2) reports a different, implemented repair — ARPACK switched to shift-invert so it converges on all seven cases, verified against LAPACK — which closes the fairness gate deterministically. As v1 anticipated, this alone does not rescue the overall verdict: the sole remaining cause is `factorized_sextic_ground`’s speed, now attributed specifically to $k = 1$ and resolved by narrowing the verdict’s scope to $k > 1$, not by overturning the $k = 1$ result.

T5. Novelty of retention is not established. Section 17 positions bracket inheritance against nested iteration and cascadic multigrid (Bornemann and Deuffhard, 1996; Trottenberg et al., 2001) and identifies two candidate differences, but we have run no comparison against a cascadic eigenvalue solver. Until that exists, the architectural claim is a positioning statement, not a demonstrated advance.

T6. “Raw input, no tuning” overstates the interface. The planner is free of *user* tuning, not of tuning. The paths described here contain at least fifteen fixed numeric constants—the 1025-point probe, the initial radius, the $8s$ half-width with its floor and cap, the 0.05ϵ decay threshold, the $\kappa^{-1/4}$ scale, the factor 8 and floor 64 in (29), the 0.45ϵ mesh gate, the 0.01ϵ working width and its 2×10^{-12} floor, the $2s$ and 0.1 -width margins, the bisection iteration cap, and the 10^{-300} pivot floor. Each was fixed once by the author; none has a published sensitivity analysis. A referee is entitled to ask what happens at 0.9ϵ instead of 0.45ϵ , and we cannot answer.

T7. Comparator selection. The primary peer is DSTEZ. The state of the art for requested-only tridiagonal eigenvalues is MRRR (Dhillon and Parlett, 2004), and we report no MRRR timings—only the methodological note of Section 17 about why the f2py route cannot be timed. Beating the second-best

routine while declining to measure against the best is a legitimate objection, and the honest answer is that the MRRR arm has not been run here.

T8. The declared suite is benign — addressed, with two losses. All seven declared potentials are smooth and well conditioned. Section 16 evaluates outside that regime and finds the boundaries: the kernel loses by up to $3.1\times$ on block-splittable matrices (Section 16.1), a single kink triples the solve count (Section 16.2), and the planner returns a confidently ACCEPTed answer that is wrong by 16.07 on a high-barrier double well (Section 16.3). Still untested: Coulomb and other singular potentials, discontinuous V , and matrices requiring the scaling half of DSTEBZ’s hardening (Demmel et al., 1995).

T9. The boundary gate is unsound, not merely uncertified — demonstrated. Remark 2 showed the decay score is anti-conservative; Section 16.3 shows it is worse than that. Because the score tests a local property of the boundary point, a barrier separating two wells satisfies it exactly as a decaying tail does, and the window witness inherits the same blind spot. This is a soundness failure with a live counterexample, not a missing proof. Any use of the word “certified” is excluded until the gate establishes that the forbidden region extends to infinity; the repair is sketched in Section 16.3 and is not implemented here.

T10. Richardson extrapolation is conditional; the bound is an estimator. Expansion (30) is reasonable for smooth potentials. Table 2 shows B_j exceeds the audited error on all seven instances, but by a stable factor, which identifies it as an error estimator with an empirical safety margin rather than a bound (Section 12.1). Neither result extends to non-smooth or singular potentials or to near-degenerate clusters.

T11. Reproducibility record — improved, still incomplete. Table 9 records the replication host in full and Run D uses three process launches. Two gaps remain: the recorded run’s own `processor` field is `unknown`, so Run A cannot be attributed to specific hardware; and the replication host has one vCPU (Table 9), so a regression that would appear only when a multithreaded LAPACK is available to the comparator cannot be detected here. Runs C and D should be repeated on a multi-core host.

19 Conclusion

Starting from raw problem data alone, the Retained Multilevel Sturm architecture computed the declared low-lying spectra of seven one-dimensional Schrödinger operators within their declared tolerances, with the reference values withheld from every

planning decision. On the harmonic instance the maximum audited deviation is 1.465×10^{-10} against a tolerance of 2×10^{-8} , released under a diagnostic bound of 2.540×10^{-9} ; across all seven the released bound exceeds the audited error by factors of 7.7 to 21.4, which identifies it as a well-behaved estimator rather than a proved bound.

The larger part of this paper is the correction of its own benchmark, and the corrections were not free. Holding the comparator to the same bisection half-width reduces the kernel-only geometric mean from 2.560 to 1.908. Replacing the unpaired eleven-sample single-launch design with interleaved arms, a paired bootstrap and 150 pairs across three launches reduces it further to 1.867 and, more importantly, converts one instance into a reproducible *loss* to DSTEBZ, so the project’s own speed verdict falls from ACCEPT to HOLD. (Section 15.10, added in v2, traces this specific instance to `factorized_sextic_ground`’s $k = 1$ and reports that under a scope corrected to $k > 1$ the verdict is ACCEPT in five independent runs out of five; the $k = 1$ result itself is not overturned, only excluded from a claim it cannot support.) Porting retention into the end-to-end comparator—giving it the idea the architecture is proposing—reduces the end-to-end geometric mean from 3.279 to 2.411. Two attempts to dismiss the remaining advantage fail: it does not decay with problem size up to $n \approx 8 \times 10^5$, and the diagnostic gate is never violated on the suite.

Evaluated outside the declared suite the boundaries appear quickly. The kernel’s advantage inverts on matrices its opponent can split into blocks— $3.1\times$ slower on a glued Wilkinson matrix with all eigenvalues requested—because its advantage comes from batching brackets through one long sweep and block splitting makes the sweep short. And on a symmetric double well whose central barrier rises above the requested levels, the raw-input planner returns the spectrum of a single well, with a passing decay gate, a passing window witness, status ACCEPT, and an error of 16.07 against a tolerance of 2×10^{-8} . The gate asks whether the boundary lies in a classically forbidden region; a barrier answers yes exactly as a tail does. That is a soundness failure in the component the architecture is named for, found by the first potential we built to look for one, and it was invisible to a declared suite whose only double well requests a single level.

What survives is narrower than what was originally claimed and, we think, more

correct computation on seven smooth spectra
+ a fair end-to-end advantage of $2.4\times$
+ a kernel that loses where blocks split
+ a truncation gate that is unsound
+ a novelty claim not yet established

(44)

and not a universal claim of superiority over SciPy. The honest scope of the planner, until the gate of

Section 16.3 is repaired, is single-well potentials with monotone tails.

A Pipeline in pseudocode

Algorithm 1 Retained Multilevel Sturm

Require: $(V, \theta, k, \varepsilon)$

- 1: $x_* \leftarrow$ minimiser of V on the probe grid
- 2: $\kappa \leftarrow$ second central difference of V at x_* ; $s \leftarrow \kappa^{-1/4}$
- 3: $w_0 \leftarrow \min(\max(8s, 2), 16)$; $[a_0, b_0] \leftarrow [x_* - w_0, x_* + w_0]$
- 4: **if** decay score $D < D_{\min}$ **then** widen $[a_0, b_0]$ and repeat
- 5: $M \leftarrow M_0$; brackets $\leftarrow$ Gershgorin bounds
- 6: **repeat**
- 7: assemble H_M from (20)
- 8: **for** each active j : validate inherited bracket by Sturm count, doubling r_j or falling back to global bounds
- 9: batched bisection on (21)–(22) to width τ_E
- 10: $M \leftarrow 2M$; inherit brackets
- 11: **until** three meshes available and $\max_j \Delta_{\text{mesh},j} \leq 0.45\varepsilon$
- 12: $m \leftarrow \max(2s, 0.1(b_0 - a_0))$; $[a_1, b_1] \leftarrow [a_0 - m, b_0 + m]$
- 13: $M_{\text{expanded}} \leftarrow M(b_1 - a_1)/(b_0 - a_0)$; one raw solve
- 14: $\Delta_{\text{window},j}$ by (36); E_j by (37)
- 15: **return** E_j with status ACCEPT iff $\max_j(\Delta_{\text{mesh},j} + \Delta_{\text{window},j}) \leq \varepsilon$

B Recorded configuration

C Replication host

D The Run C change

Run C differs from Run B by the following, and by nothing else. In the end-to-end comparator:

```
eigh_tridiagonal(
    diagonal, off_diagonal,
    select="i", select_range=(0, problem.modes - 1),
    eigvals_only=True, check_finite=False,
    tol=max(problem.tolerance * 0.01, 2.0e-12), # matched
    working_half_width
    lapack_driver="stebz",
)
```

and in the kernel audit, both solvers are held to the same width:

```
def _matched_width(problem) -> float:
    return max(problem.tolerance * 0.01, 2.0e-12)

# native:
native_eigvals_from_tridiagonal(diagonal, off_diagonal, k,
    _matched_width(problem))

# comparator:
sla.eigh_tridiagonal(diagonal, off_diagonal, select="i",
    select_range=(0, k - 1), eigvals_only=
        True,
        check_finite=False,
        tol=_matched_width(problem),
        lapack_driver="stebz")
```

Passing `tol=problem.tolerance` instead—the apparently natural choice—does not terminate; see Remark 5.

Run D adds two further pieces. The paired harness times every arm once per repeat in a shuffled order and bootstraps paired repeat indices:

```
for _ in range(repeats):
    rng.shuffle(names) # arms interleaved,
    seeded
    for name in names:
        t = perf_counter(); calls[name]()
        samples[name].append(perf_counter() - t)

idx = rng.integers(0, n, size=n) # PAIRED resample
boot = median(b[idx]) / median(a[idx])
```

and the retained comparator arm replaces the second full Richardson ladder by one same-spacing solve plus transport of the correction already purchased—i.e. (37), with the window search, decay gate, mesh ladder and DSTEBZ calls left untouched:

```
ratio = (exp[1] - exp[0]) / (win[1] - win[0])
n_exp = max(64, int(round(finest * ratio)))
raw_exp = solve(problem, exp, n_exp) # ONE solve
shift = abs(raw_exp - raw_fine)
values = richardson_fine + (raw_exp - raw_fine)
```

Reproducibility statement

Run A quantities are read from the artefact’s committed results file. Runs B, C and D were produced by executing that same committed code on the host of Table 9: Run C with the single change of Section D, Run D additionally with the paired-timing harness and the retained comparator arm of Section D. Every figure in Tables 2 and 4 to 6 comes from a run executed for this paper, not copied from a prior report. All timing figures are host-specific; the outstanding reproducibility gaps—the recorded run’s unset processor field, the absence of multiple independent process launches, and the single-core replication host—are recorded under T7 and T8 of Section 18.

Added in v2: the ARPACK shift-invert repair of Section 15.10 is commit 72d5eec, with the 10^{-9} – 10^{-13} LAPACK-agreement figure and the ARPACK-timing-inflation caveat both from the independent re-verification of commit acf4b51 (pull request #109). The instruction-count instrument and the $k > 1$ scope correction are commits 43c0ac4 and 3823245 (pull request #110), which passed two rounds of independent review with all continuous-integration checks green before merge. Both pull requests are merged to main at commit 439480d at the time of this writing.

Data and code availability

The engine, the comparator pipeline, the same-operator audit, the bootstrap routines, and the recorded results file are distributed as part of the open-source *Information Discrete Mathematics* artefact under the MIT licence, at <https://github.com/morrocwi/information-discrete-math> (commits and pull requests cited in this paper are resolvable at that repository). The single-line change defining Run C is given verbatim in Section D, so

Table 8: Declared constants of the recorded run.

Quantity	Value
Tolerance ε	2×10^{-8}
Levels k	4
Probe grid	1025 points, initial radius 8
Initial window	$[-8, 8]$
Verification window	$[-10, 10]$
Mesh ladder (initial window)	128, 256, 512, 1024, 2048, 4096
Expanded solve	$M = 5120$ (dimension 5119)
Bisection width τ_E	2×10^{-10}
Mesh gate	0.45ε
Pivot floor	10^{-300}
Kernel benchmark size	$M = 768, n = 767, k = 4$
Kernel / end-to-end samples	11 / 21
Confidence intervals	percentile bootstrap, 95%

Table 9: Replication host for Runs B and C. Thread variables are pinned by the harness before NumPy, SciPy, or Numba import, so the comparison measures algorithms rather than the host thread scheduler.

Item	Value
CPU	Intel Xeon @ 2.10 GHz, 1 vCPU, 1 thread/core
RAM	3 GiB
Kernel	Linux 6.18.5, glibc 2.39
Python	3.12.3
NumPy / SciPy	2.4.4 / 1.17.1
Numba / llvmlite	0.66.0 / 0.48.0
BLAS/LAPACK	OpenBLAS, 1 thread
Native kernel	compiled (<code>native_kernel_compiled = true</code>)
JAX	not installed (JAX comparator skipped)
Thread pinning	all BLAS/Numba thread vars = 1
Repeats (Runs B, C)	21 end-to-end, 11 kernel
Repeats (Run D)	15 e2e, 50 kernel, $\times 3$ launches
Bootstrap	4000 resamples, seed 20260727

the controlled experiment of Section 15 can be reconstructed from the published sources without further information. The v2 additions of Section 15.10 (the ARPACK repair, the `k1_discrete_readout` module, and the corresponding tests) are in the same artefact and licence, at the commits given above. This manuscript and its prose documentation are CC-BY-4.0; a showcase repository collecting the paper source (both this manuscript and v1), benchmark scripts, and a cross-reference from every claim in this paper to the script and data file that produced it is at <https://github.com/morroccwi/retained-sturm>.

Use of AI assistance

AI-based writing assistance was used for language editing and for structuring the manuscript. All mathematical derivations, numerical results, interpretation, and the identification of the defects reported in Section 18 are the responsibility of the author, who has verified every equation and every reported figure. The v2 material of Section 15.10 was produced with AI-assisted implementation and analysis (the ARPACK repair, the instruction-count instrument, and the corrected retention-mechanism claim), checked at each

step against the actual implementation and independently reviewed twice before inclusion here; responsibility for the final claims remains the author’s, per the same standard as the rest of this statement.

Competing interests

The author declares no competing interests.

Acknowledgements

The author thanks Walancha Supantarika.

References

- E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen. *LAPACK Users’ Guide*. SIAM, Philadelphia, 3rd edition, 1999.
- P. B. Bailey, W. N. Everitt, and A. Zettl. Algorithm 810: The SLEIGN2 Sturm–Liouville code. *ACM*

Table 10: Recorded run (Run A) environment, as committed. The `processor` field is unset, which is the gap recorded under T7.

Item	Value
Python	3.13.13
NumPy / SciPy	2.4.6 / 1.17.1
Numba	0.65.1
JAX / jaxlib	0.11.0 / 0.11.0
Platform	Linux 7.0.0, glibc 2.43
Processor	unknown

- Transactions on Mathematical Software*, 27(2):143–192, 2001.
- W. Barth, R. S. Martin, and J. H. Wilkinson. Calculation of the eigenvalues of a symmetric tridiagonal matrix by the method of bisection. *Numerische Mathematik*, 9:386–393, 1967.
- F. A. Bornemann and P. Deuffhard. The cascadic multigrid method for elliptic problems. *Numerische Mathematik*, 75(2):135–152, 1996.
- D. T. Colbert and W. H. Miller. A novel discrete variable representation for quantum mechanical reactive scattering via the S -matrix Kohn method. *Journal of Chemical Physics*, 96(3):1982–1991, 1992.
- J. W. Demmel, I. Dhillon, and H. Ren. On the correctness of some bisection-like parallel eigenvalue algorithms in floating point arithmetic. *Electronic Transactions on Numerical Analysis*, 3:116–149, 1995. (LAPACK Working Note 115.)
- I. S. Dhillon and B. N. Parlett. Multiple representations to compute orthogonal eigenvectors of symmetric tridiagonal matrices. *Linear Algebra and its Applications*, 387:1–28, 2004.
- B. Efron. Bootstrap methods: another look at the jackknife. *Annals of Statistics*, 7(1):1–26, 1979.
- S. K. Lam, A. Pitrou, and S. Seibert. Numba: a LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15)*, pages 1–6. ACM, 2015.
- V. Ledoux, M. Van Daele, and G. Vanden Berghe. MATSLISE: A MATLAB package for the numerical solution of Sturm–Liouville and Schrödinger equations. *ACM Transactions on Mathematical Software*, 31(4):532–554, 2005.
- B. N. Parlett. *The Symmetric Eigenvalue Problem*. Classics in Applied Mathematics 20. SIAM, Philadelphia, 1998.
- S. Pruess and C. T. Fulton. Mathematical software for Sturm–Liouville problems. *ACM Transactions on Mathematical Software*, 19(3):360–376, 1993.
- J. D. Pryce. *Numerical Solution of Sturm–Liouville Problems*. Oxford University Press, Oxford, 1993.
- L. F. Richardson. The approximate arithmetical solution by finite differences of physical problems involving differential equations. *Philosophical Transactions of the Royal Society A*, 210:307–357, 1911.
- U. Trottenberg, C. W. Oosterlee, and A. Schüller. *Multigrid*. Academic Press, London, 2001.
- P. Virtanen et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.
- J. H. Wilkinson. *The Algebraic Eigenvalue Problem*. Clarendon Press, Oxford, 1965.